

SI-Clash 2025



Digital Track Final Report

Team 4

Team Members:

Menna Reda Mohamed
Manar Saber Abdelrahim
Eslam Elsayed Elwehedy Elshokafy
Maria Adel Ramzy

Mentor: ENG.Youssef Gamal

2025

Contents

Abstract	4
List of Figures	5
List of Tables	6
1 Introduction	7
1.1 Problem Overview	7
1.2 Motivation	7
1.3 Project Objectives	7
1.4 Key Contributions	7
2 Background and Theory	7
2.1 Digital Sampling Theory and Frequency Analysis	8
2.2 Multirate Signal Processing Fundamentals	8
2.2.1 Decimation and Interpolation	8
2.2.2 Polyphase Decomposition	8
2.3 IIR Biquads and Notch Filtering (Contextual)	8
2.4 CIC Filter Theory and Properties	8
2.5 Fixed-Point Arithmetic, Quantization, and Word Growth	9
3 Fractional Polyphase Decimator Design ($L/M = 2/3$)	9
3.1 Design Strategy	9
3.1.1 The Standard Approach (Interpolation-First)	9
3.1.2 The Optimized Approach (Decimation-First)	10
3.2 Proposed Architecture for $L/M = 2/3$	10
3.2.1 Structural Description	11
3.3 Implementation Results	11
3.3.1 Functional Verification	12
3.3.2 Resource Utilization and Optimization	12
3.3.3 Timing Analysis	13
4 IIR Notch Filters (2.4 MHz and 5.0 MHz)	13
4.1 System Specifications	13
4.2 Fixed-Point Implementation Analysis	13
4.3 Filter 1: 2.4 MHz Notch	15
4.3.1 Notch Analysis & Optimization	15
4.3.2 System Level Verification	17
4.3.3 FPGA Implementation Results: Timing and Utilization	18
4.3.4 Data Flow	19
4.3.5 Accuracy Verification	19
4.4 Filter 2: 5.0 MHz Notch	19
4.4.1 Analysis & Optimization	19
4.4.2 System Level Verification	21
4.4.3 FPGA Implementation Results: Timing and Utilization	21
4.4.4 Data Flow	22

4.4.5	Accuracy Verification	22
5	CIC Decimator and Compensation FIR	23
5.1	CIC Architecture and Rationale	23
5.2	Bit-Growth and Saturation Handling	24
5.3	Compensation FIR Design	24
5.4	Run-time Configurability (1, 2, 4, 8, 16)	25
5.5	Resource and Timing Optimization	26
5.5.1	FPGA Implementation Results: Timing and Utilization	26
5.5.2	Data Flow	27
5.5.3	Accuracy Verification	27
6	Future Work	27
A	Code Listings	28
	References	28

Abstract

This thesis presents the design, implementation, and verification of a multistage Digital Front End (DFE) for radio and ADC preprocessing, targeting efficient sample-rate conversion, interference suppression, and hardware-optimized digital filtering. The system processes an incoming 9 MHz digitized RF/IF signal and produces a spectrally clean output compatible with downstream demodulation stages. The proposed architecture consists of three primary components: a fractional polyphase FIR decimator for 9-to-6 MHz conversion, two cascaded second-order IIR notch filters for attenuating narrowband interferers at 2.4 MHz and 5.0 MHz, and a configurable CIC decimator supporting factors of 1, 2, 4, 8, and 16.

Filter coefficient generation, quantization, and preliminary frequency-response validation were performed using MATLAB, including the use of Simulink-to-HDL workflows where appropriate. All RTL modules were implemented in Verilog and SystemVerilog, with emphasis on streaming throughput, deterministic latency, and fixed-point arithmetic suitable for FPGA or ASIC deployment. The fractional decimator meets the required passband ripple (≤ 0.25 dB) and achieves at least 80 dB stopband attenuation. The IIR notches were realized using a stable direct-form II transposed structure, providing at least 50 dB rejection at each interferer frequency. The CIC stage incorporates an optional FIR compensation filter to correct passband droop and ensure a final ripple of ≤ 0.5 dB across all decimation factors.

A comprehensive verification environment was developed using a Python-based golden model, enabling sample-by-sample comparison between floating-point reference outputs and fixed-point RTL behavior. Automated SystemVerilog testbenches validated frequency response, aliasing performance, stability, overflow handling, and full-rate throughput at the 9 MHz input rate. Results demonstrate that the implemented DFE satisfies all numerical and architectural specifications while maintaining low latency and robust fixed-point performance, making it a suitable solution for modern digital communication receivers and programmable radio systems.

All source code, RTL implementations, Python golden models, and verification scripts related to this project are publicly available at:

<https://github.com/mennareda84/DFE-design>

List of Figures

1	Standard model for SRC by a rational factor L/M (Interpolation followed by Decimation).	9
2	Equivalent Networks to implement the Optimized Design.	10
3	Efficient polyphase SRC for $L/M = 2/3$. Note the input commutation (decimation) occurs prior to filtering.	11
4	Resampling Visualization: Input at 9 MHz vs Output at 6 MHz. The output correctly reconstructs the signal at the lower rate.	12
5	Post-Implementation Resource Utilization Summary.	12
9	Design Timing Summary showing positive slack (WNS = 6.231 ns).	13
10	Block Diagram of IIR Form II Transposed	14
11	Response of 16-bit Design to 2.4 MHz Input (Insufficient Attenuation)	15
12	Response of 16-bit Design to 2.4018 MHz Input (Notch is Shifted)	16
13	Response of Optimized 20-bit Design to 2.4 MHz Input (Target Met)	17
14	Filter Selectivity Test: Recovery of 200 kHz Signal from 2.4 MHz Noise	18
15	Vivado Timing Summary (Constraints Met)	18
16	FPGA Resource Utilization Report	19
17	Synthesized Data Flow View in Vivado	19
18	RMS Comparison	19
19	20-bit Design Response: Stability Check and Attenuation	20
20	Filter Selectivity Test: Recovery of 200 kHz Signal from 5 MHz Noise	21
21	Vivado Timing Summary (All Constraints Met)	22
22	FPGA Resource Utilization Report	22
23	Synthesized Data Flow View in Vivado	22
24	RMS Comparison (Golden Model vs RTL)	23
25	Vivado Timing Summary (Constraints Met)	26
26	FPGA Resource Utilization Report	26
27	Synthesized Data Flow View in Vivado	27
28	RMS Comparison	27

List of Tables

1 Cascaded Notch Filter Specifications 13

1 Introduction

1.1 Problem Overview

Modern digital communication and signal-processing systems frequently require converting a signal from one sampling rate to another. When the desired conversion ratio is fractional—such as $2/3$ —the process becomes more complex because it must maintain signal quality while managing hardware constraints. This project implements a fractional decimation filter based on a polyphase FIR structure, targeting efficient FPGA deployment.

1.2 Motivation

Fractional rate conversion is essential in wireless receivers, software-defined radios, audio resampling, and multi-standard communication platforms. Hardware implementations must meet strict requirements: high throughput, minimal resource usage, numerical stability, and predictable timing. Creating a complete design—from DSP theory to fixed-point hardware—is a strong practical demonstration of real-world digital design.

1.3 Project Objectives

This project aims to:

- Design a fractional decimator ($L = 2$, $M = 3$) using a polyphase FIR approach.
- Convert the floating-point DSP model into a hardware-friendly fixed-point format.
- Implement the design in Verilog and target FPGA synthesis.
- Validate the design using a golden-model reference and a testbench with RMS-error evaluation.
- Verify performance metrics such as passband ripple, stopband attenuation, group delay, and sampling-rate conversion accuracy.

1.4 Key Contributions

The project provides:

- A complete RTL implementation of a fractional decimator.
- A fixed-point processing pipeline optimized for FPGA efficiency and accuracy.
- A Python-based golden model that generates stimulus vectors and expected outputs.
- Full frequency-response and RMS-error verification aligned with the given specification.

2 Background and Theory

This section introduces the theoretical foundations required to design and implement a fractional sampling-rate converter. The material covers essential DSP concepts, multirate signal-processing theory, fixed-point numerical behavior, and filtering structures used throughout the project.

2.1 Digital Sampling Theory and Frequency Analysis

Discrete-time signals are described by sampling a continuous signal at a rate F_s . Understanding how frequency components behave under sampling relies on the Discrete-Time Fourier Transform (DTFT) for mathematical analysis, and the Discrete Fourier Transform (DFT) for practical numerical evaluation.

The DTFT provides a continuous frequency representation of a discrete sequence, while the DFT gives a sampled version suitable for implementation. These tools are fundamental for evaluating passband shape, aliasing behavior, and stopband attenuation of the decimator.

2.2 Multirate Signal Processing Fundamentals

Multirate processing refers to systems where signals change sampling rates internally. It provides a computationally efficient way to manipulate bandwidth, support multi-standard receivers, and reduce processing cost.

2.2.1 Decimation and Interpolation

Decimation reduces the sampling rate by an integer factor M by inserting filters that suppress aliasing. Interpolation increases the sampling rate by a factor L by inserting zero-valued samples and applying a smoothing filter. Fractional conversion combines the two operations. In this project, the rate changes from F_s to $\frac{2}{3}F_s$ using:

$$\text{Interpolation: } L = 2, \quad \text{Decimation: } M = 3.$$

2.2.2 Polyphase Decomposition

Direct FIR filtering at the upsampled rate is computationally costly. Polyphase decomposition reorganizes the FIR filter into sub-filters that operate at the lower input or output rate. For a filter of length N , it splits the impulse response into M or L branches depending on the structure. For the fractional converter, the filter is decomposed into six polyphase branches corresponding to the (L, M) combination. This approach reduces the computation rate and enables efficient FPGA implementation.

2.3 IIR Biquads and Notch Filtering (Contextual)

Although the main design uses a FIR polyphase filter, many communication receivers include IIR biquad sections or notch filters to mitigate narrowband interference. A brief understanding of these structures provides context for comparing FIR- and IIR-based designs, especially in terms of stability, quantization sensitivity, and frequency selectivity.

2.4 CIC Filter Theory and Properties

Cascaded Integrator–Comb (CIC) filters are commonly used in integer-rate sample converters. They are multiplier-free structures offering very low hardware cost. Although the fractional SRC in this project does not use CIC filters directly, their properties illustrate why FIR-based polyphase designs are preferred when precise passband shaping and high stopband attenuation (e.g., 80 dB) are required.

2.5 Fixed-Point Arithmetic, Quantization, and Word Growth

Hardware implementations rely on fixed-point arithmetic where signals and coefficients are represented with predefined word lengths (e.g., s16.15 or s1.19). Quantization introduces rounding error, coefficient truncation error, and internal overflow risks. Word growth occurs naturally in FIR accumulation due to multiple products being summed:

$$\text{Word Growth} \approx \log_2(N) + \text{input bits} + \text{coefficient bits}.$$

Managing these effects ensures numerical stability, prevents overflow, and preserves the required filter performance after implementation on an FPGA.

3 Fractional Polyphase Decimator Design ($L/M = 2/3$)

3.1 Design Strategy

To achieve the required sampling rate conversion from 9 MHz to 6 MHz, we utilize a rational factor of $L/M = 2/3$, where $L = 2$ and $M = 3$. While high-level software tools provide generic functions for this conversion, a direct hardware implementation of those algorithms yields suboptimal efficiency. Our design approach leverages Noble Identities and Polyphase Decomposition to rearrange the signal flow graph, prioritizing arithmetic efficiency by placing decimation-style structures before the filtering operations and finally interpolation.

3.1.1 The Standard Approach (Interpolation-First)

In standard Digital Signal Processing (DSP) literature, a rational sampling rate conversion is typically modeled as an interpolation followed by decimation as shown in Figure 1.

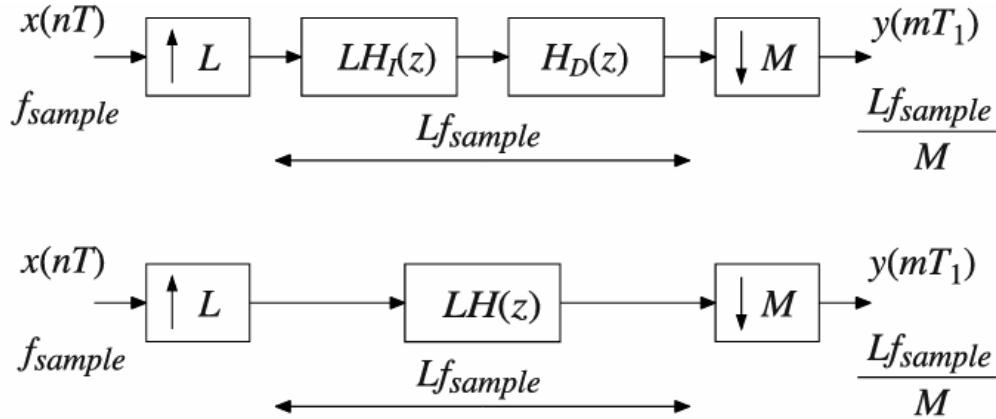


Figure 1: Standard model for SRC by a rational factor L/M (Interpolation followed by Decimation).

- **The Process:** The input signal $x(nT)$ is first up-sampled by a factor of L , filtered by a single anti-imaging/anti-aliasing filter $H(z)$, and finally down-sampled by a factor of M .

- **Software Implementation:** This topology corresponds to the standard realization used by tools such as MATLAB's `resample` function.
- **Inefficiency:** In this model, the filtering occurs at the intermediate frequency $L \cdot f_{sample}$. For our specific case ($L = 2$), this would require the hardware to process data at 18 MHz. Furthermore, the up-sampling process inserts zero-valued samples, meaning a significant portion of the arithmetic operations involve redundant multiplications by zero. But it is used because it is generic whatever the values of L and M .

3.1.2 The Optimized Approach (Decimation-First)

To optimize the design for FPGA implementation, we utilize polyphase decompositions that operate at the lowest sampling frequency. By applying **Noble identities**, we can rearrange the structure such that the “**decimation**” (in the form of input commutation) occurs first.

Since the interpolation factor $L = 2$ and decimation factor $M = 3$ are relatively **prime integers** (they share no common integer factor), we are mathematically permitted to interchange the ordering of the up-sampler and down-sampler. This allows us to move the filtering operations to a **lower sampling rate domain** before up-sampling occurs. These steps are illustrated in Figure 2

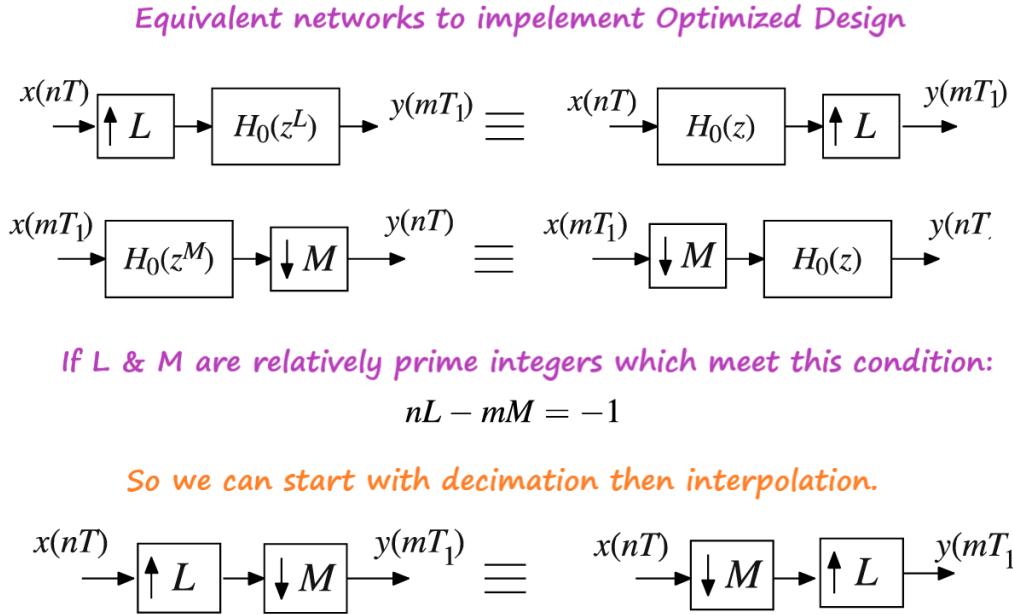


Figure 2: Equivalent Networks to implement the Optimized Design.

3.2 Proposed Architecture for $L/M = 2/3$

Based on the derivations for rational converters, we implement the specific network shown in Figure 3. This structure is the efficient polyphase sampling rate converter specifically derived for the $2/3$ ratio.

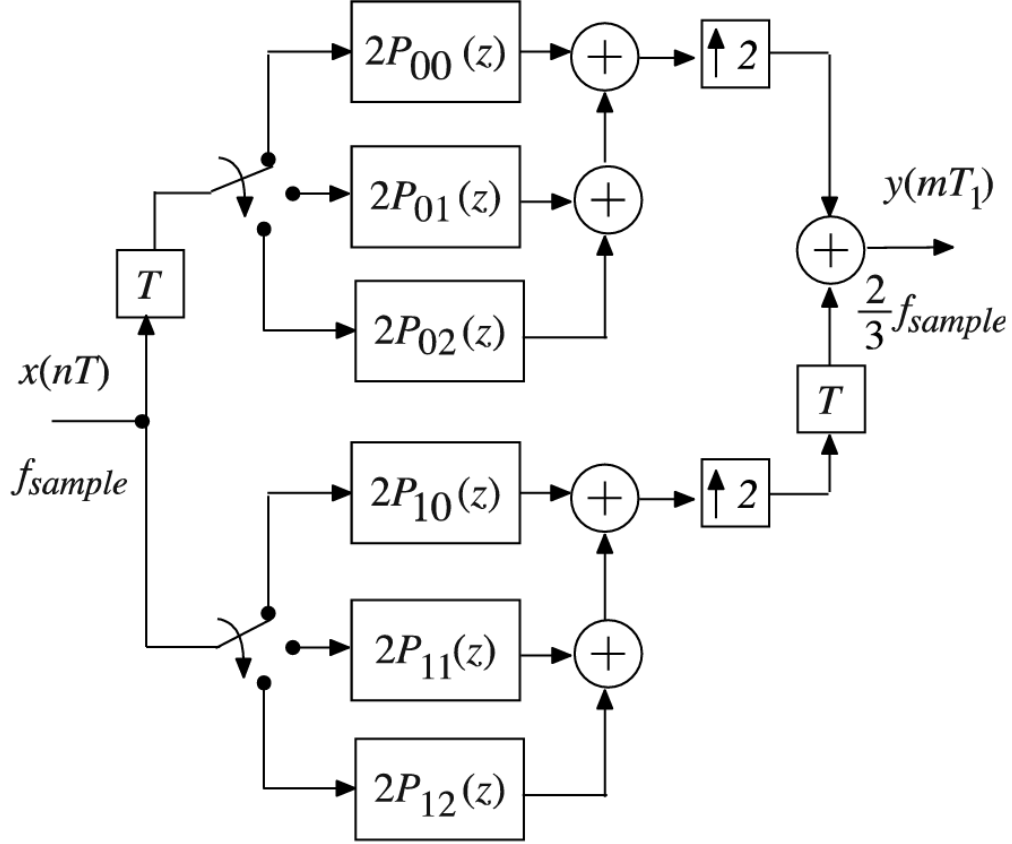


Figure 3: Efficient polyphase SRC for $L/M = 2/3$. Note the input commutation (decimation) occurs prior to filtering.

3.2.1 Structural Description

The architecture consists of three distinct stages:

1. **Input Commutation (Decimation Stage):** The input delay chain and down-sampling operations are arranged to act as a commutator. This distributes the input samples into parallel polyphase branches immediately, effectively lowering the data rate at the input.
2. **Polyphase Filtering:** The signal is processed by parallel sub-filters (e.g., $2P_{00}(z)$, $2P_{01}(z)$). Because the rate reduction occurred at the input, these filters operate at a reduced sampling frequency, avoiding the calculation of zero-valued samples.
3. **Output Summation (Interpolation Stage):** The outputs of the parallel branches are summed and up-sampled to construct the final 6 MHz output $y(mT_1)$.

This architecture ensures that the delays on the input and output sides, which operate at different sampling rates due to $l_0 = 1$ and $m_0 = 1$, are correctly synchronized. This results in the highest computational efficiency for this specific conversion ratio.

3.3 Implementation Results

3.3.1 Functional Verification

To validate the correctness of the fractional rate conversion, the design was simulated with a test stimulus. Figure 4 illustrates the time-domain response of the converter.

- **Input:** A sinusoidal waveform sampled at 9 MSPS (Blue trace).
- **Output:** The resulting waveform resampled at 6 MSPS (Orange trace).

As observed, the output samples correctly track the envelope of the input signal. The fractional delay is handled correctly by the polyphase structure, preserving the signal integrity without introducing phase distortion or amplitude modulation artifacts.

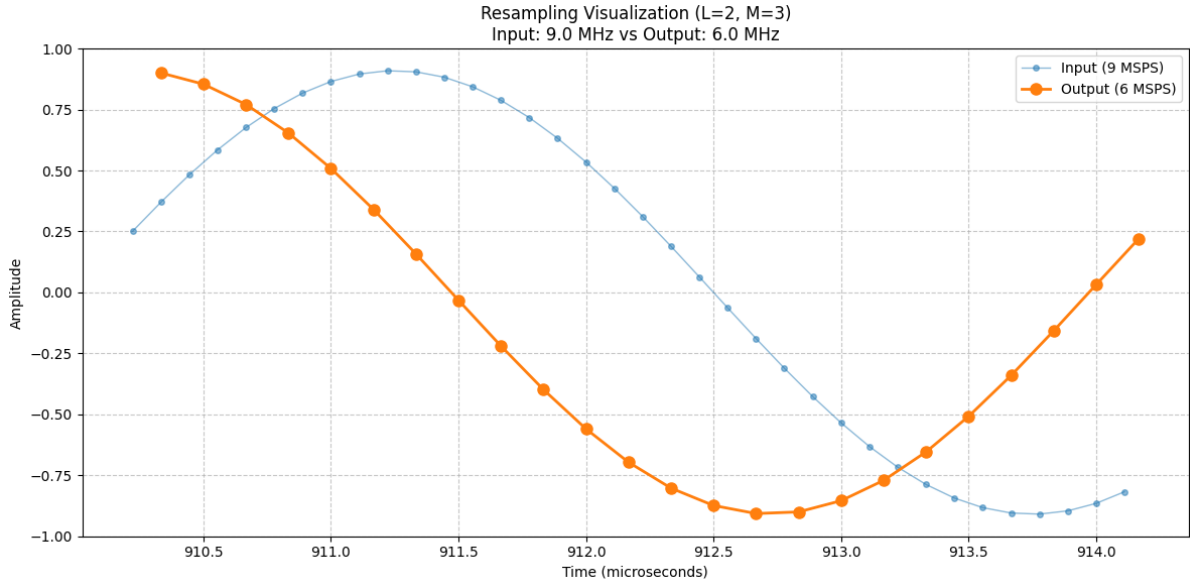


Figure 4: Resampling Visualization: Input at 9 MHz vs Output at 6 MHz. The output correctly reconstructs the signal at the lower rate.

3.3.2 Resource Utilization and Optimization

The design was synthesized and implemented targeting the FPGA fabric. The resource utilization summary is presented in Figure 5 .

Exploiting Coefficient Symmetry: A key factor in achieving this efficiency is the exploitation of the linear-phase FIR filter’s coefficient symmetry. By folding the delay line and pre-adding symmetric data samples before multiplication, the architecture significantly reduces the required number of DSP48 slices. Without this optimization, the number of multipliers would be approximately double the current count.

Name	^1	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	LUT as Memory (46200)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
FinalFractionalDecimator		2681	5037	1377	2643	38	72	38	1
u_dsphdl_FIRRateConverter (dsphdl_FIRRateConverter)		2681	5037	1377	2643	38	72	0	0

Figure 5: Post-Implementation Resource Utilization Summary.

3.3.3 Timing Analysis

Static timing analysis confirms that the design meets all timing constraints with a positive margin. As shown in Figure 9, the design achieves a Worst Negative Slack (WNS) of 6.231 ns. This substantial timing margin indicates that the pipelining within the polyphase branches is sufficient to support the required clock frequencies without critical path violations.

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS): 6.231 ns		Worst Hold Slack (WHS): 0.086 ns	Worst Pulse Width Slack (WPWS): 4.230 ns
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9011		Total Number of Endpoints: 9011	Total Number of Endpoints: 5148
All user specified timing constraints are met.			

Figure 9: Design Timing Summary showing positive slack (WNS = 6.231 ns).

4 IIR Notch Filters (2.4 MHz and 5.0 MHz)

4.1 System Specifications

The system requires two cascaded second-order Infinite Impulse Response (IIR) notch filters (biquads) to suppress narrowband interferers. The architecture is Direct Form II Transposed (DF2T) to minimize latency and improve fixed-point stability.

Table 1: Cascaded Notch Filter Specifications

Parameter	Value
Sampling Frequency (F_s)	6.0 MHz
Notch 1 Center Frequency	2.4 MHz
Notch 2 Center Frequency	5.0 MHz
Attenuation Goal	≥ 60 dB per notch
Bandwidth (-3 dB)	< 100 kHz (Narrow)
Architecture	Cascaded DF2T Biquads
Latency	1–2 sample periods per stage
I/O Precision	Signed 16-bit (Q1.15)
Coefficient Precision	Signed 20-bit (Q2.18) [Optimized]

4.2 Fixed-Point Implementation Analysis

The fixed-point implementation followed a systematic design flow, moving from high-level modeling to optimized hardware realization. The methodology is detailed as follows:

- **Design and Architectural Modeling:** The process began with the **MATLAB Filter Designer** tool to calculate the initial coefficients and verify the theoretical frequency response.

Subsequently, a **Simulink block diagram** was constructed as shown in Figure 10 to model the Direct Form II Transposed architecture, allowing for a clear analysis of the signal flow and arithmetic operations.

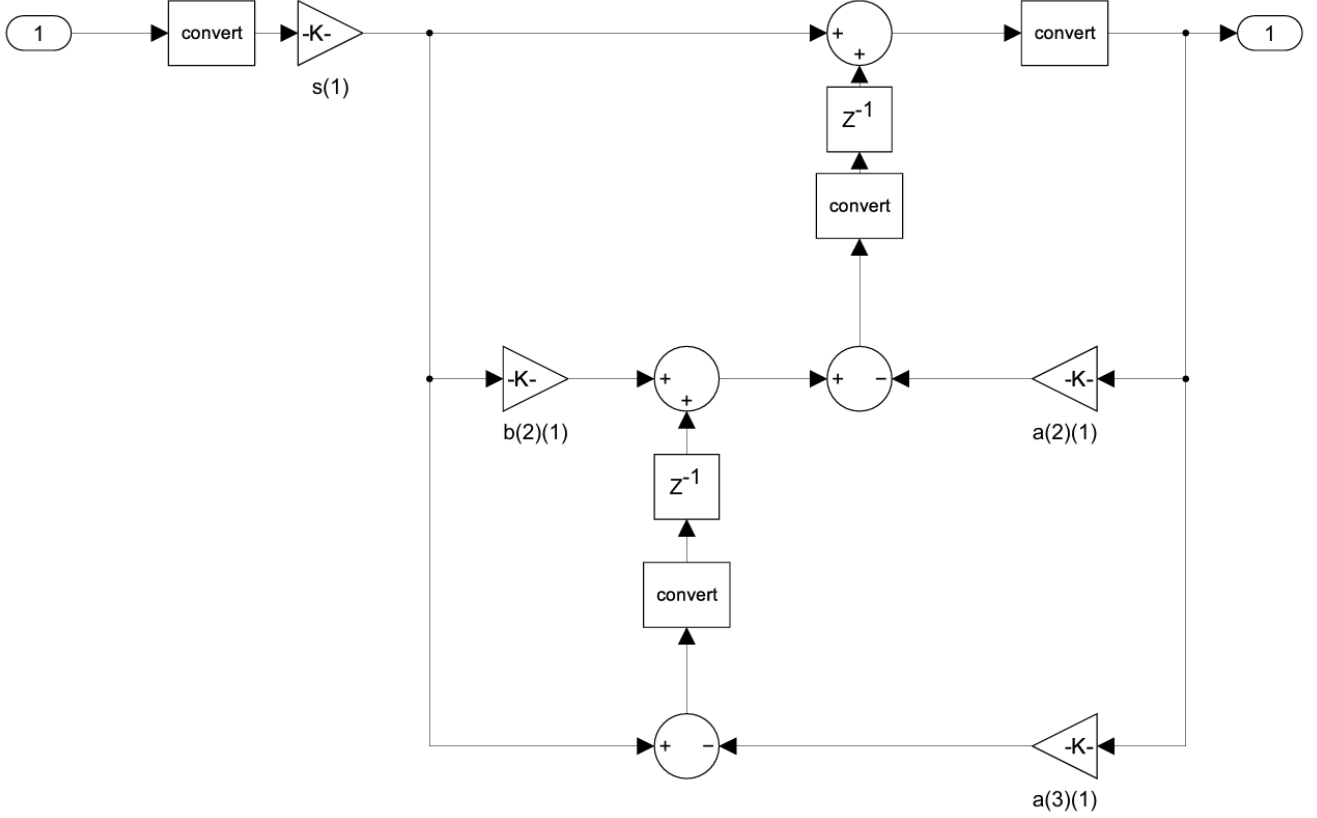


Figure 10: Block Diagram of IIR Form II Transposed

- **Baseline Code Generation:** The initial HDL code was generated using **MATLAB HDL Coder**. This approach was chosen because the tool generates code with optimized structures for resource utilization and timing closure. This provided a robust and efficient architectural baseline to learn from and build upon.
- **Accuracy Optimization:** While the auto-generated code was resource-efficient, it required arithmetic refinement to meet the strict spectral specifications. The design was iteratively improved by manually increasing coefficient precision to 20 bits and fine-tuning the integer values to overcome quantization grid errors. This allowed the design to achieve the required ≥ 60 dB attenuation.
- **Zero-Error Verification:** The final optimized design was validated against a bit-accurate Python Golden Model. This methodology ensured that the hardware implementation maintained the timing benefits of the generated code while achieving **zero error** in output accuracy compared to the reference model.

4.3 Filter 1: 2.4 MHz Notch

4.3.1 Notch Analysis & Optimization

The target notch frequency is 2.4 MHz. With a sampling rate of $F_s = 6.0$ MHz, the ideal angle is $\theta = 144^\circ$.

Phase 1: 16-bit Coefficient Limitation The initial design utilized 16-bit coefficients (Q2.14). Simulation with a 2.4 MHz test tone revealed a significant performance deficiency. As shown in Figure 11, the filter achieved only ≈ 28.5 dB of attenuation, failing the 60 dB requirement.

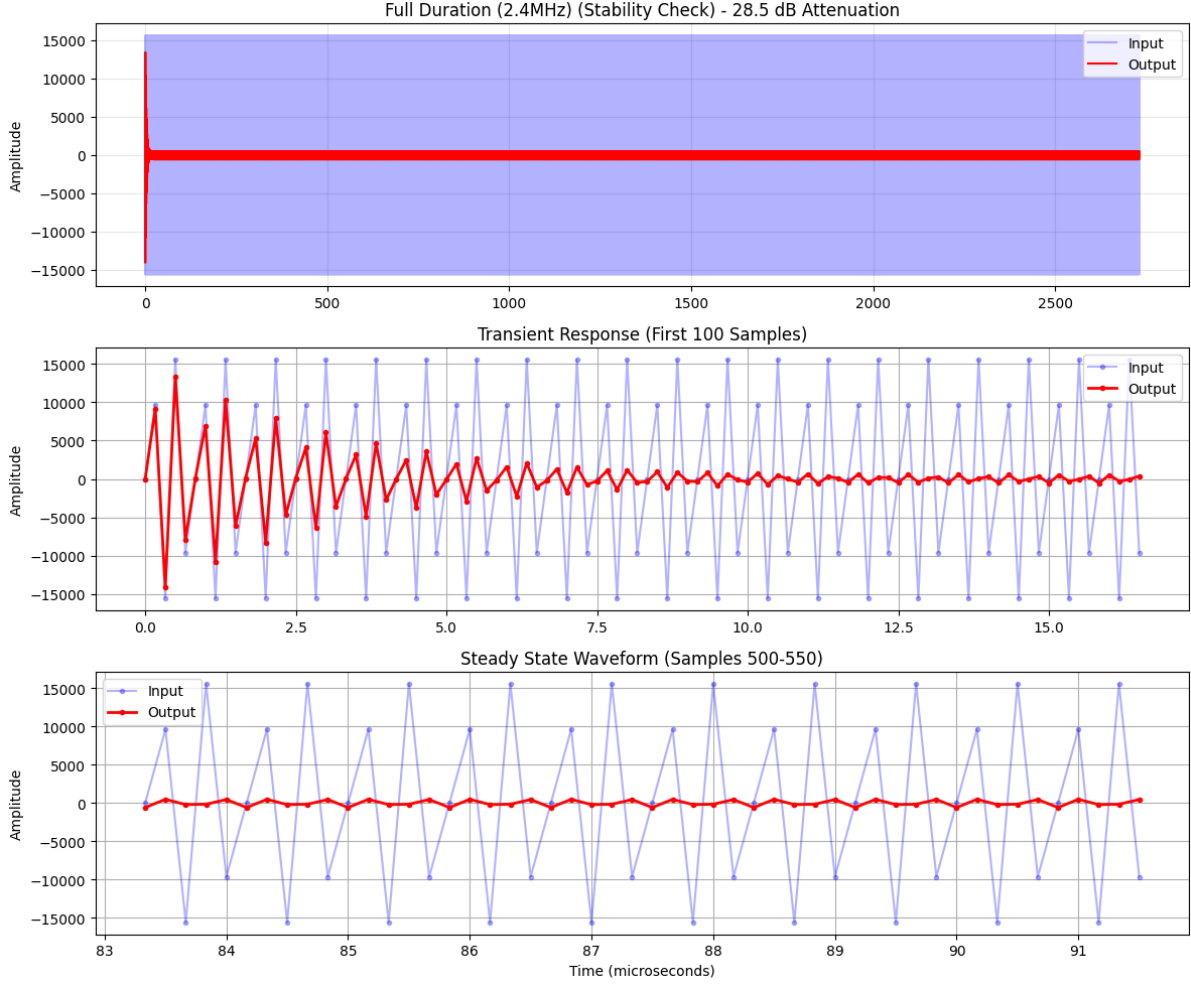


Figure 11: Response of 16-bit Design to 2.4 MHz Input (Insufficient Attenuation)

Root Cause Analysis The poor attenuation was traced to coefficient quantization. The 16-bit grid forced the coefficient b_1 to a value that shifted the actual notch center to approximately **2.4018 MHz**. When tested with a signal at exactly 2.4018 MHz (Figure 12), the filter achieved high attenuation (≈ 68 dB), proving the architecture was correct but the frequency alignment was off.

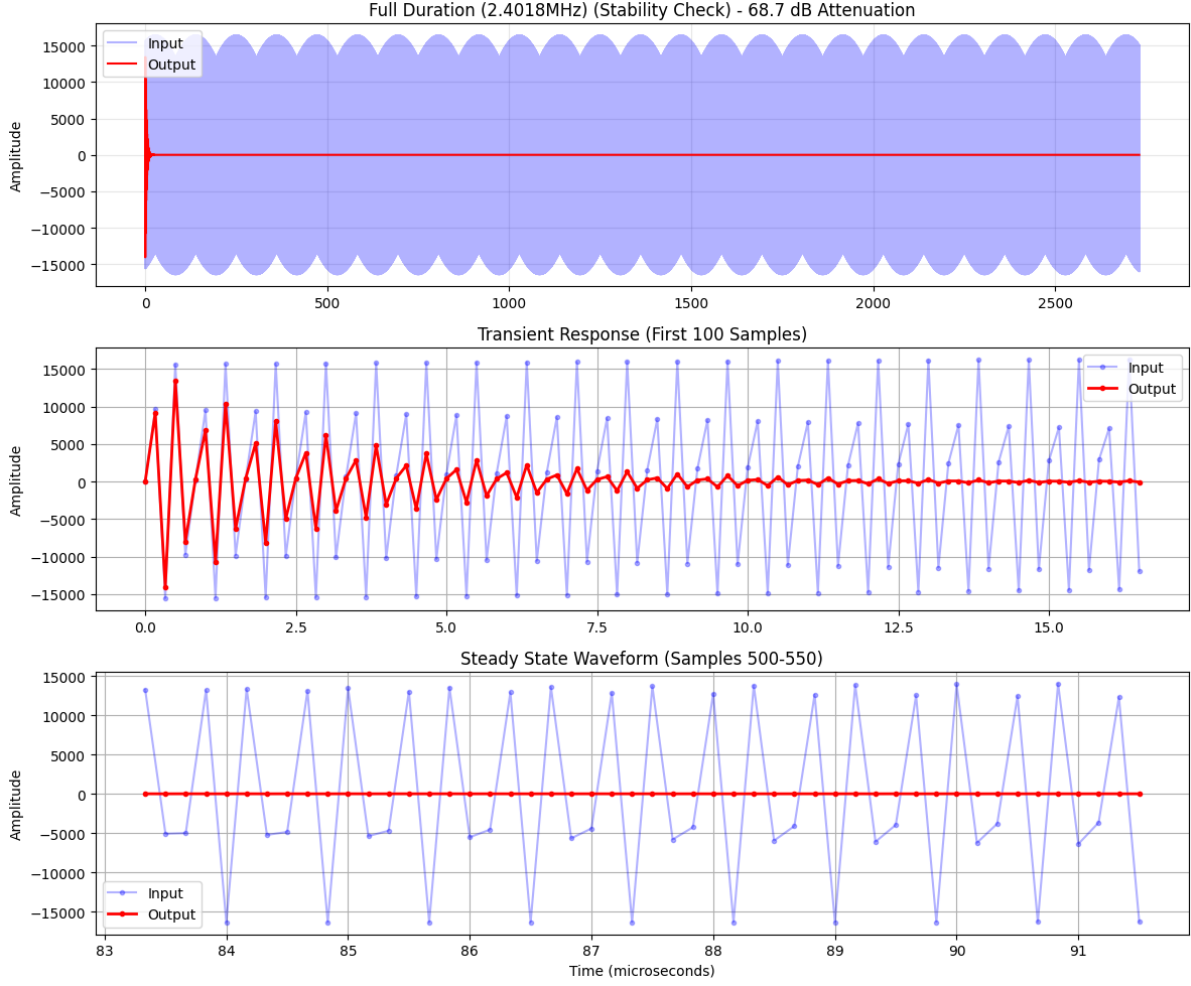


Figure 12: Response of 16-bit Design to 2.4018 MHz Input (Notch is Shifted)

Phase 2: 20-bit Optimization To resolve the frequency shift, the coefficient precision was increased to 20-bit (Q2.18). The final coefficient was manually tuned by calculating the ideal value and rounding to the nearest 20-bit integer:

$$b_{1,ideal} = -2 \cos(144^\circ) \times 2^{18} \approx 424158 \quad (1)$$

Using this optimized value, the notch aligned perfectly with the target frequency. Figure 13 demonstrates the final response to a 2.4 MHz input, achieving stable attenuation of > 74 dB.

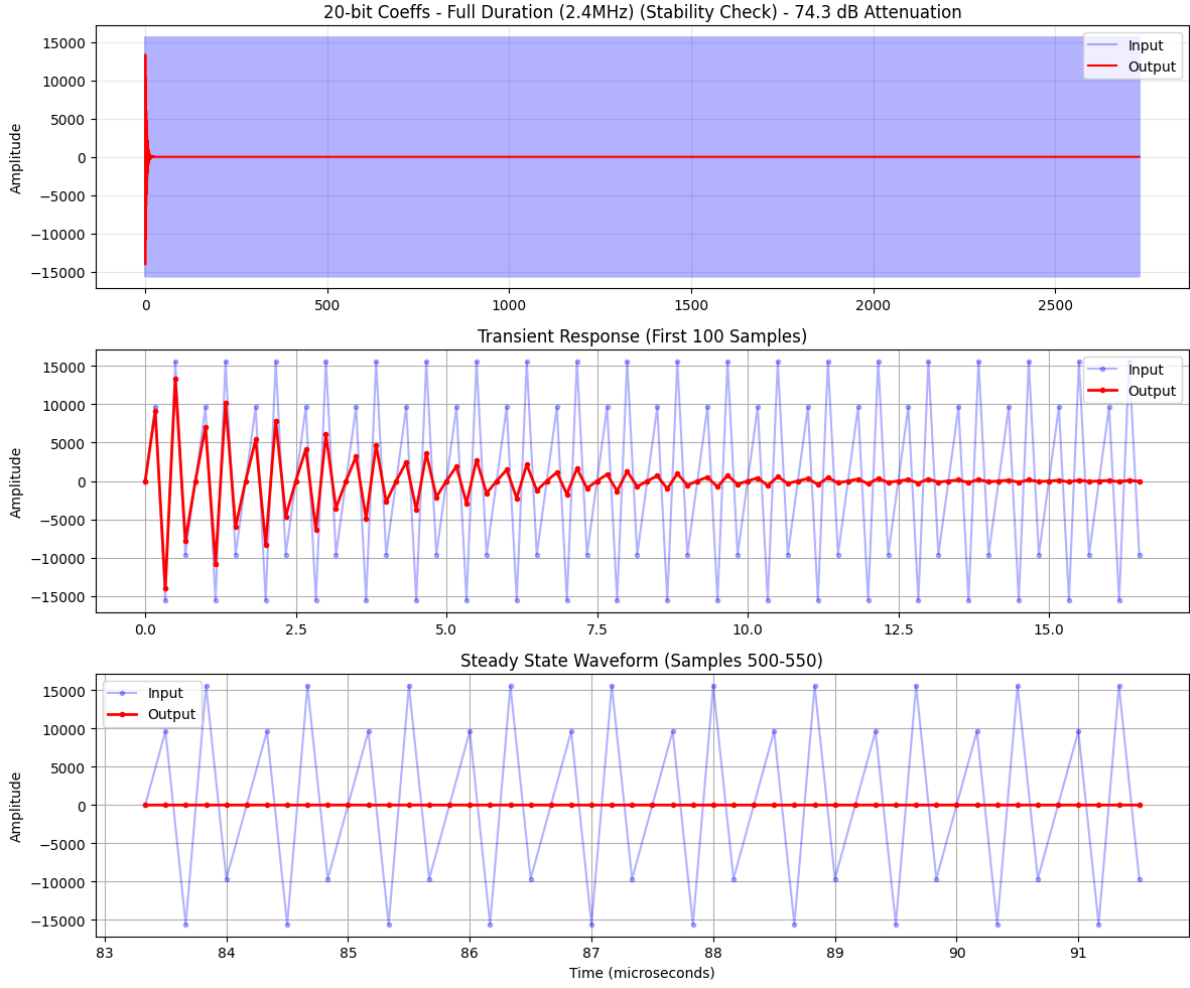


Figure 13: Response of Optimized 20-bit Design to 2.4 MHz Input (Target Met)

4.3.2 System Level Verification

To validate the filter's selectivity, a composite test signal was generated consisting of a desired 200 kHz sine wave added to a high-amplitude 2.4 MHz noise interferer.

Figure 14 illustrates the time-domain results. The top plot shows the input signal, heavily distorted by the high-frequency noise. The bottom plot shows the filter output, where the 2.4 MHz noise has been completely removed, recovering the clean 200 kHz desired signal.

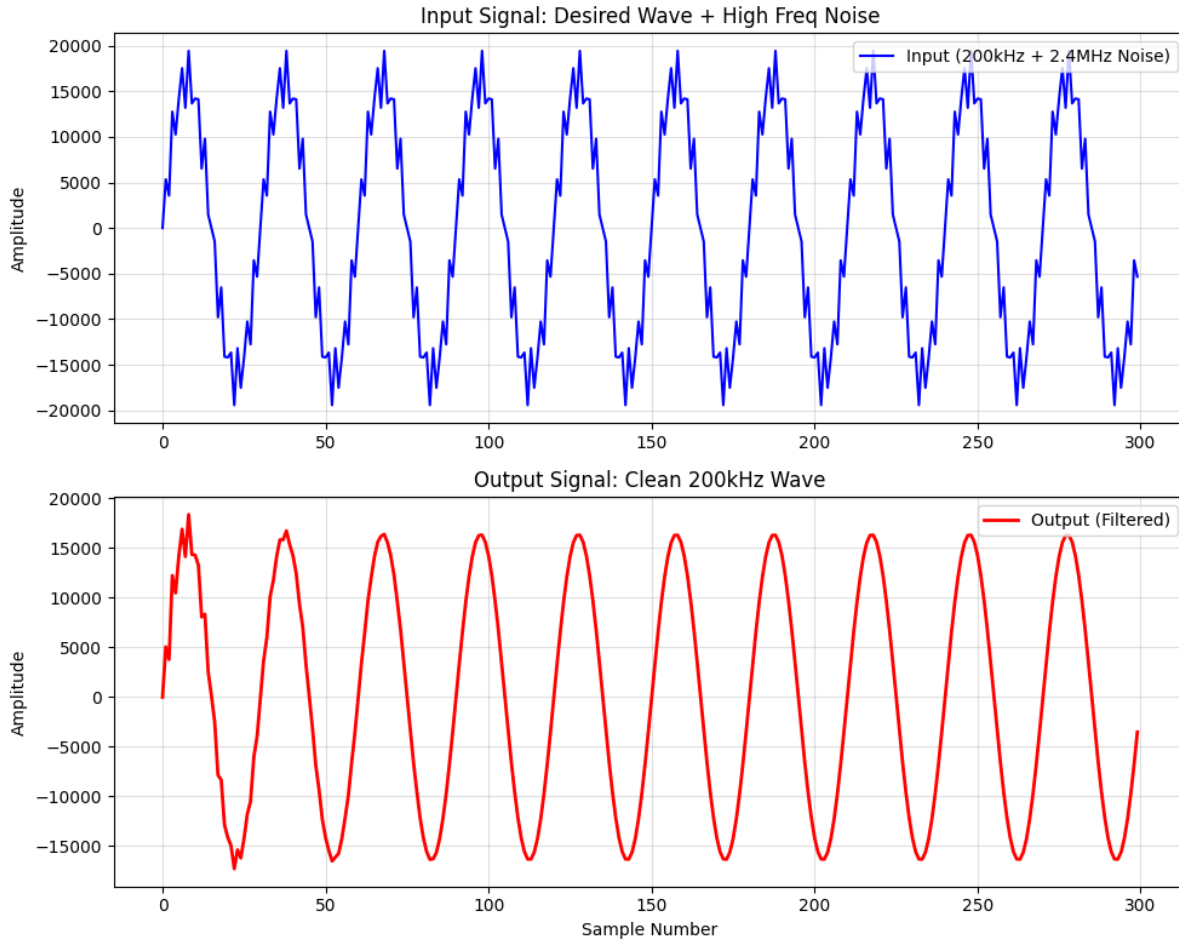


Figure 14: Filter Selectivity Test: Recovery of 200 kHz Signal from 2.4 MHz Noise

The design was synthesized and implemented in Xilinx Vivado. The following metrics confirm the hardware feasibility and performance.

4.3.3 FPGA Implementation Results: Timing and Utilization

The design successfully met all timing constraints with a positive slack (WNS: 0.113 ns), indicating the pipelining strategy was effective for the 6 MHz target clock (Figure 15). Resource utilization remained low (Figure 16), consuming only 80 LUTs and 5 DSP slices, making it highly efficient for integration into larger systems.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.113 ns	Worst Hold Slack (WHS): 0.126 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 285	Total Number of Endpoints: 285	Total Number of Endpoints: 112

All user specified timing constraints are met.

Figure 15: Vivado Timing Summary (Constraints Met)

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	DSPs (90)	Bonded IOB (106)	BUFGCTRL (32)
Final20Notch24	80	106	48	80	5	38	1
u_dsphdl_BiquadFilter (dsphdl_BiquadFilter)	80	106	48	80	5	0	0

Figure 16: FPGA Resource Utilization Report

4.3.4 Data Flow

The synthesized data flow (Figure 17) showing the clear path from input registers through the DSP blocks to the output.

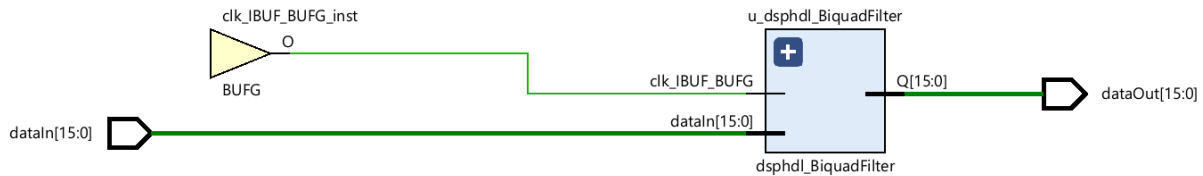


Figure 17: Synthesized Data Flow View in Vivado

4.3.5 Accuracy Verification

Finally, the hardware output was compared against the bit-accurate Python Golden Model. As shown in Figure 18, the RMS error is exactly **0.00**, confirming that the RTL implementation is a bit-perfect match to the theoretical model.

```

...   === RMS Comparison ===
      Golden RMS    = 11591.52
      RTL RMS       = 11591.52
      RMS Error     = 0.00
      Relative Err  = 0.0000%

```

Figure 18: RMS Comparison

4.4 Filter 2: 5.0 MHz Notch

4.4.1 Analysis & Optimization

Note on Coefficient Precision: Unlike the previous 2.4 MHz design, this iteration bypassed the 16-bit development phase. The design was implemented directly using 20-bit coefficients (Q2.18) to ensure maximum precision from the outset.

Mathematical Justification for Immediate Success The 5.0 MHz notch filter exhibited superior performance immediately, achieving high attenuation without requiring manual tuning. This stability is due to the specific mathematical relationship between the target frequency and the sampling rate.

The target frequency of 5.0 MHz is aliased to 1.0 MHz relative to the Nyquist zone ($6.0 - 5.0 = 1.0$ MHz). This creates a frequency ratio of exactly $1/6$:

$$\theta_{5MHz} = 360^\circ \times \frac{1.0}{6.0} = 60^\circ \quad (2)$$

Calculating the ideal coefficient b_1 :

$$b_{1,ideal} = -2 \cos(60^\circ) = -2 \times (0.5) = -1.0 \quad (3)$$

Since -1.0 can be represented exactly in binary fixed-point arithmetic (corresponding to -2^{18} in Q2.18 format), there is **zero quantization error** in the frequency location. Consequently, the filter center aligns perfectly with the target.

Figure 19 demonstrates the impulse and transient response, confirming the filter's stability and high attenuation capabilities (> 74 dB as shown in simulation).

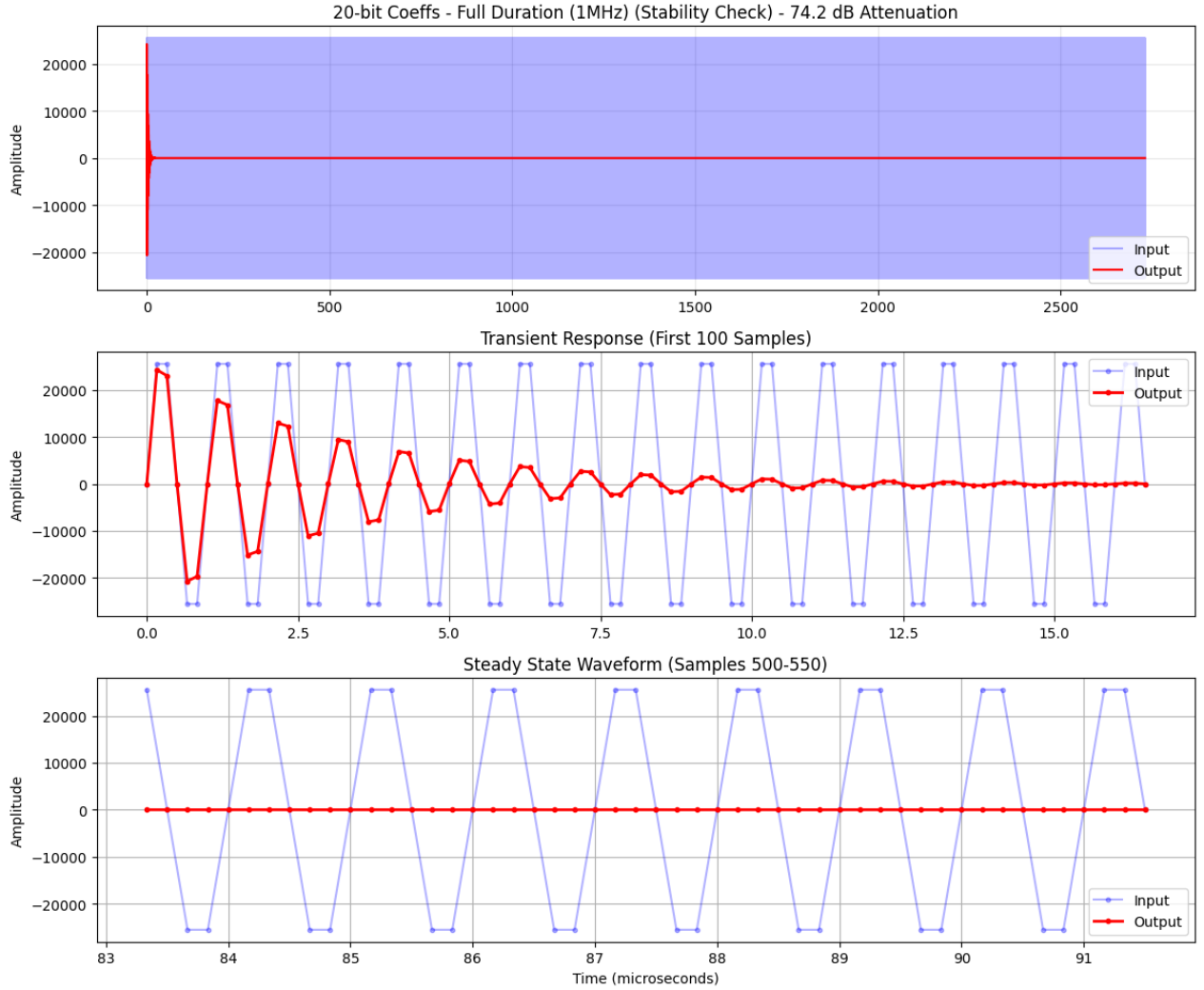


Figure 19: 20-bit Design Response: Stability Check and Attenuation

4.4.2 System Level Verification

To verify selectivity, the filter was tested against a composite signal containing a 200 kHz desired wave mixed with 5 MHz high-frequency noise.

As shown in Figure 20, the input signal (blue) is heavily distorted. The filter successfully suppresses the 5 MHz noise component, yielding a clean 200 kHz output sine wave (red), demonstrating the effectiveness of the exact coefficient alignment.

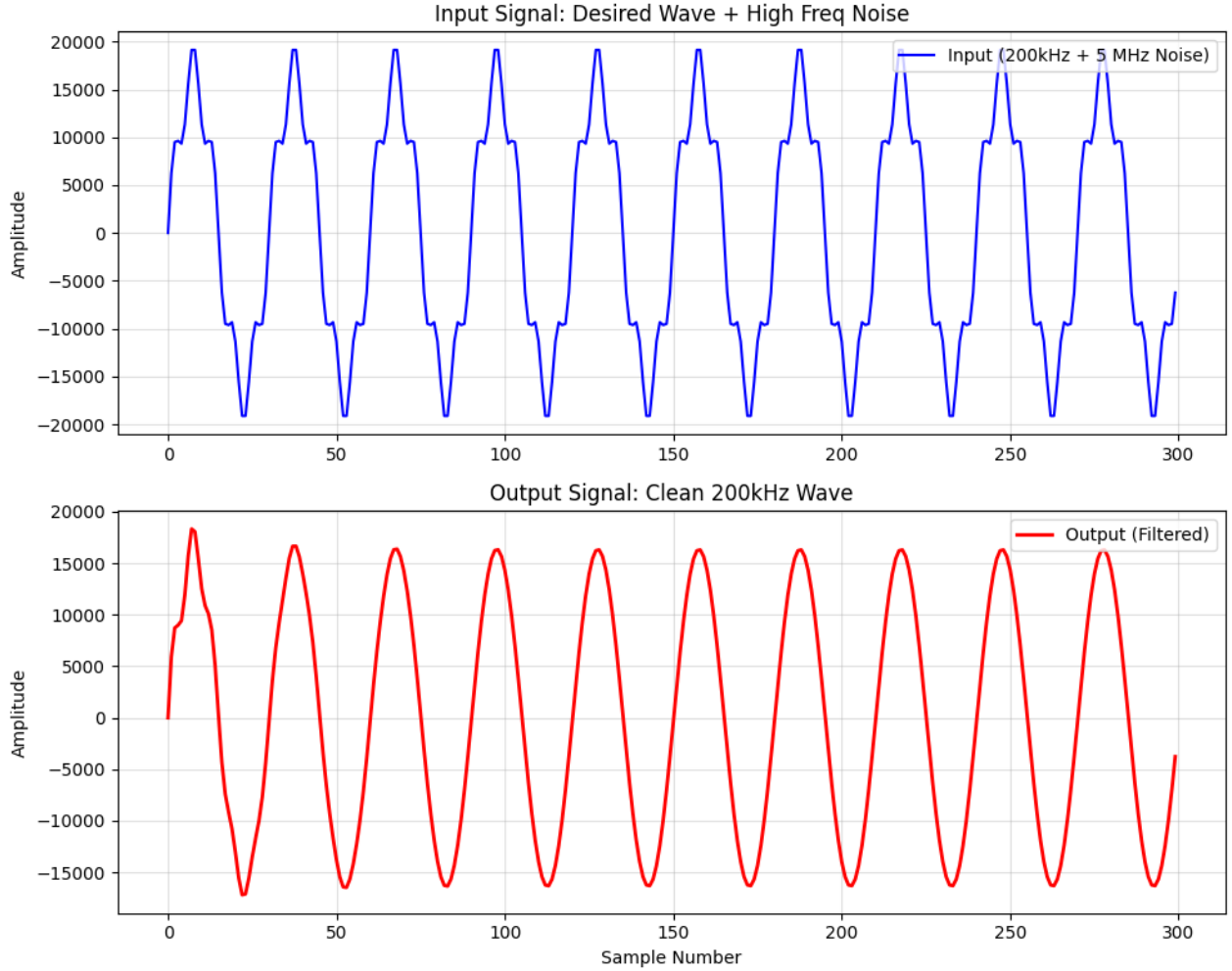


Figure 20: Filter Selectivity Test: Recovery of 200 kHz Signal from 5 MHz Noise

4.4.3 FPGA Implementation Results: Timing and Utilization

The design was synthesized for the target FPGA. The timing summary (Figure 21) indicates a robust implementation with a Worst Negative Slack (WNS) of 0.113 ns.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.113 ns	Worst Hold Slack (WHS): 0.126 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 285	Total Number of Endpoints: 285	Total Number of Endpoints: 112

All user specified timing constraints are met.

Figure 21: Vivado Timing Summary (All Constraints Met)

Resource utilization (Figure 22) remains consistent with the previous filter architecture, utilizing 80 LUTs and 5 DSP slices, confirming that the change in notch frequency did not impact hardware complexity.

Name	^1	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	DSPs (90)	Bonded IOB (106)	BUFGCTRL (32)
Final20Notch5		80	106	48	80	5	38	1
> u_dsphdL_BiquadFilter (dsphdL_BiquadFilter)		80	106	48	80	5	0	0

Figure 22: FPGA Resource Utilization Report

4.4.4 Data Flow

The synthesized data flow (Figure 23) visualizes the pipeline stages and DSP instantiation for the 5 MHz architecture.

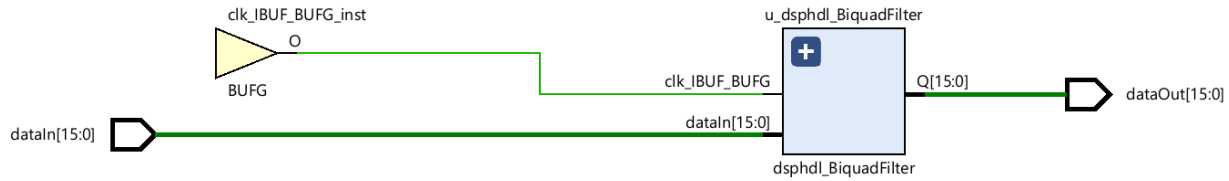


Figure 23: Synthesized Data Flow View in Vivado

4.4.5 Accuracy Verification

The RTL output was compared against the Golden Model. Figure 24 confirms bit-perfect accuracy with an RMS error of 0.00.

=== RMS Comparison ===	
Golden RMS	= 502.99
RTL RMS	= 502.99
RMS Error	= 0.00
Relative Err	= 0.0000%

Figure 24: RMS Comparison (Golden Model vs RTL)

5 CIC Decimator and Compensation FIR

5.1 CIC Architecture and Rationale

The Cascaded Integrator–Comb (CIC) decimator in the DFE provides low-complexity, high-throughput sample-rate reduction while requiring only adders and registers. The implementation follows the classical Hogenauer structure, consisting of an N -stage integrator chain operating at the high input sample rate, followed by a rate change and an N -stage comb differentiator operating at the reduced output rate.

In this design, the CIC filter uses `STAGES` = 3 and supports decimation factors $R \in \{1, 2, 4, 8, 16\}$, selected dynamically via the `d_sel` control input. The integrator chain is implemented as a series of accumulators:

$$y_0[n] = y_0[n-1] + x[n], \quad y_i[n] = y_i[n-1] + y_{i-1}[n], \quad i = 1, \dots, N-1.$$

These operate at the full input rate, ensuring continuous streaming and preventing data loss. The maximum CIC gain is

$$G_{\max} = R^N,$$

which motivates the internal word-width growth handled later in Section 6.2.

Decimation is performed using a rate counter. Once every R input samples, the output of the final integrator is captured into a snapshot register:

$$\text{snap_reg} \leftarrow y_{N-1}[n],$$

and a one-cycle strobe triggers comb processing. The comb stages, implemented as

$$h_0[n] = \text{snap_reg}[n] - \text{delay}_0[n-1], \quad h_i[n] = h_{i-1}[n] - \text{delay}_i[n-1],$$

run only at the decimated rate, significantly reducing power and resource usage.

The CIC decimator therefore provides a highly efficient hardware architecture: integrators at the high rate maintain simple adder-only updates, while the comb stages operate at a much lower rate, reducing switching activity by up to $16\times$. This structure also guarantees deterministic timing, making it suitable for FPGA/ASIC implementations and continuous-stream radio front-ends. An optional FIR compensation filter is included to correct the passband droop inherent to CIC responses.

5.2 Bit-Growth and Saturation Handling

CIC filters exhibit substantial internal bit-growth due to repeated accumulation in the integrator stages. The theoretical worst-case growth is

$$\Delta B = N \log_2(R),$$

and for $R_{\max} = 16$ and $N = 3$:

$$\Delta B = 3 \times 4 = 12 \text{ bits.}$$

In the implementation, the internal width is computed as:

$$\text{INT_W} = \text{IN_W} + \Delta B + \text{HEADROOM}.$$

With $\text{IN_W} = 16$, $\text{HEADROOM} = 2$, and $\Delta B = 12$, the internal width becomes

$$\text{INT_W} = 30 \text{ bits.}$$

Both integrator and comb paths use the full INT_W width to prevent overflow and maintain numerical stability. After comb processing, the signal is still represented in 30-bit fixed-point precision and must be safely reduced back to the 16-bit output width. Direct truncation risks wrap-around artifacts; instead, a saturating truncation function is used:

$$\text{out} = \begin{cases} \text{MAX}_{16}, & \text{if } x > \text{MAX}_{16} \\ \text{MIN}_{16}, & \text{if } x < \text{MIN}_{16} \\ \text{trunc}_{16}(x), & \text{otherwise.} \end{cases}$$

This ensures that the output remains within valid 16-bit signed bounds and downstream blocks never encounter overflow or two's-complement wrap-around.

The compensation FIR follows the same philosophy. After its MAC operation, the accumulator undergoes rounding, controlled shifting, and final saturation to 16 bits. Together, these mechanisms ensure that the CIC+FIR chain is safe under full-scale input conditions, robust against worst-case transients, and numerically stable.

5.3 Compensation FIR Design

CIC filters inherently introduce passband droop, which becomes more severe for larger decimation factors. To restore flat passband performance, a fixed-point FIR compensation filter is applied after the comb section. The compensation FIR is implemented as a 32-tap, direct-form, fixed-point FIR whose coefficients are precomputed in MATLAB and loaded using `$readmemh`.

The FIR operates at the decimated rate, minimizing switching activity and power consumption. Internally, the MAC uses a wide accumulator:

$$\text{ACC_W} = \text{PROD_W} + \lceil \log_2(\text{FIR_TAPS}) \rceil + 2,$$

allowing accumulation without overflow. Rounding is performed by adding 2^{Q-1} prior to the right-shift operation that removes COEFF_Q fractional bits. Saturation logic then ensures the output is safely mapped into the 16-bit range.

The compensation filter is enabled by `fir_enable`. When disabled, the CIC output bypasses the FIR and undergoes saturated truncation directly. When enabled, the FIR corrects the droop and

ensures that the final passband ripple remains within the required ≤ 0.5 dB across all supported decimation settings.

Overall, the combination of the CIC decimator and compensation FIR enables efficient multi-rate processing while preserving high-quality spectral characteristics suitable for radio front-end applications.

5.4 Run-time Configurability (1, 2, 4, 8, 16)

A key requirement of the DFE architecture is the ability to adjust the decimation factor at run time without re-synthesizing or reconfiguring the hardware. The CIC module supports five decimation factors,

$$R \in \{1, 2, 4, 8, 16\},$$

selected dynamically by means of the 3-bit control signal `d_sel`. This selection is mapped internally to the corresponding decimation factor through a combinational `case` structure:

$$\text{d_sel} = 0 \rightarrow R = 1, \quad 1 \rightarrow R = 2, \quad 2 \rightarrow R = 4, \quad 3 \rightarrow R = 8, \quad 4 \rightarrow R = 16.$$

Because the CIC equations are independent of the exact value of R aside from the sampling interval, all integrator stages continue to operate at the full input rate for all configurations. The decimation factor affects only:

- the rate counter (`dec_cnt`),
- the generation of the sample strobe (`sample_strobe`),
- the snapshot interval of the final integrator value,
- and the delay spacing in the comb differentiator, implemented by storing the previous decimated samples in the `comb_delay` registers.

When a new decimation factor is selected, the hardware automatically adjusts the decimation interval using the updated R value without interrupting the data stream. No pipeline flush, reset, or reinitialization is required, enabling seamless run-time reconfiguration even while the system is actively processing samples.

The internal arithmetic width (`INT_W`) is dimensioned for the worst-case decimation factor $R_{\max} = 16$, ensuring that the filter remains overflow-safe regardless of the selected operating mode. Consequently, dynamic reconfiguration does not compromise numerical stability or require recalculation of internal widths.

The compensation FIR is integrated after the comb stage and operates at the decimated output rate. Because its clock enable is tied to the `sample_strobe`, it automatically adapts to the new rate, preserving correct timing and valid output assertions under all supported decimation settings.

Overall, the run-time reconfigurability allows the DFE to adapt to different bandwidth requirements, dynamic channel conditions, and multi-standard receiver modes while maintaining continuous streaming, stable fixed-point operation, and fully deterministic timing.

5.5 Resource and Timing Optimization

The CIC decimator and compensation FIR were designed with hardware efficiency as a primary constraint. The CIC architecture is inherently resource-minimal, consisting only of adders, subtractors, and delay registers, with no multipliers required in the integrator or comb stages. This results in extremely low LUT and DSP consumption compared to conventional FIR decimators.

To further optimize resources, the comb section operates exclusively at the decimated rate. By capturing the final integrator output through the snapshot register and triggering the comb logic only on `sample_strobe`, switching activity is reduced by up to $16\times$ for the maximum decimation factor. This significantly lowers dynamic power consumption and eases timing closure.

The compensation FIR, although multiplier-based, also benefits from reduced activity since it operates at F_s/R rather than the full input rate. Coefficient memory is implemented using distributed RAM, while the MAC pipeline is dimensioned to ensure timing closure at the target clock frequency.

The internal word widths were chosen to guarantee overflow-free operation while minimizing unnecessary bit growth. The use of a common maximum-width internal datapath (`INT_W`) across all decimation factors simplifies timing analysis and avoids reconfiguration overhead.

5.5.1 FPGA Implementation Results: Timing and Utilization

The CIC decimator design successfully met all timing constraints, achieving a positive slack (WNS: 3.980 ns), which demonstrates that the applied pipelining strategy is effective for the target 6 MHz clock. (Figure 25).

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.980 ns	Worst Hold Slack (WHS): 0.129 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 793	Total Number of Endpoints: 793	Total Number of Endpoints: 334
All user specified timing constraints are met.		

Figure 25: Vivado Timing Summary (Constraints Met)

Resource utilization (Figure 26) for the CIC decimator remained efficient, using 216 slice LUTs and 333 slice registers. The change in decimation architecture did not significantly impact hardware complexity, confirming the suitability of the design for integration into larger systems.

Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (106)	BUFGCTRL (32)
finalCIC_with_assertions	216	333	51	1
u_cic (CICDecimation)	216	333	0	0

Figure 26: FPGA Resource Utilization Report

5.5.2 Data Flow

Figure 27 presents the synthesized RTL schematic of the Configurable CIC Decimator wrapper. The schematic highlights the instantiation of the `u_cic` core, which serves as the central processing unit within the module. It also details the 16-bit data path, defined by the `dataIn` and `dataOut` signals, ensuring consistent bit-width handling. Furthermore, the diagram explicitly illustrates the rate control interface (`RIn`), which is utilized to dynamically select the decimation factor during runtime.

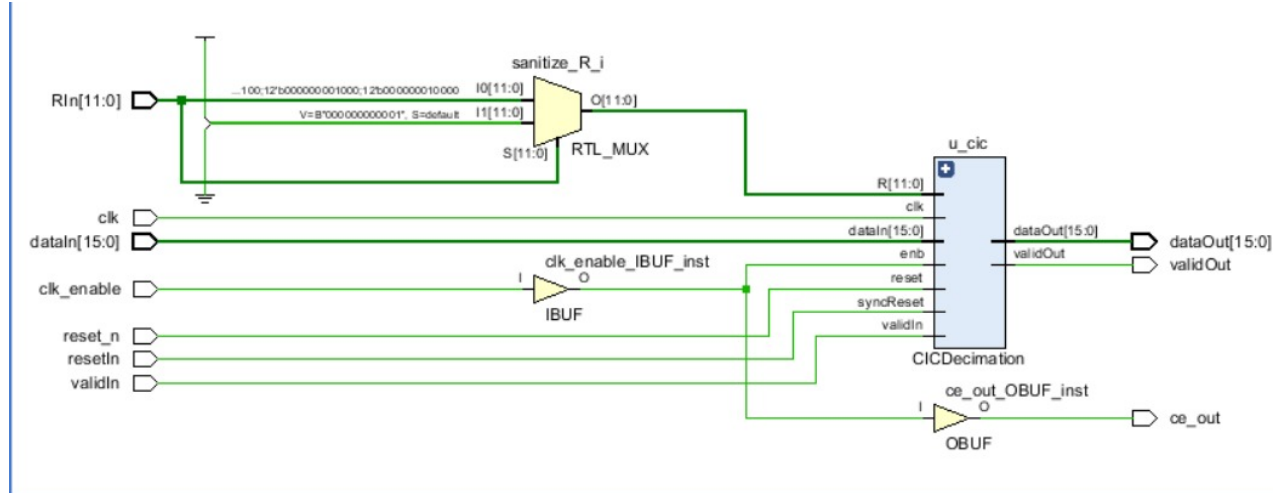


Figure 27: Synthesized Data Flow View in Vivado

5.5.3 Accuracy Verification

Finally, the hardware output was compared against the bit-accurate Python Golden Model. As shown in Figure 28.

```
=== CIC RMS Comparison (Steady-State Only) ===
Golden RMS   = 31486.08
RTL RMS      = 31486.08
RMS Error    = 0.00
Relative Err  = 0.0000%
```

Figure 28: RMS Comparison

6 Future Work

All updated work and rest of results of this project are publicly available at:
<https://github.com/mennareda84/DFE-design>

A Code Listings

All source code, RTL implementations, Python golden models, and verification scripts related to this project are publicly available at:

<https://github.com/mennareda84/DFE-design>

References

- [1] “Mixed Signal and DSP Design Techniques, 2002 — Analog Devices.” https://www.analog.com/en/resources/technical-books/mixed_signal_dsp_design_book.html
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2009.
- [3] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4th ed., McGraw-Hill, 2011.
- [4] M. Smith, *DSP for FPGA: Polyphase Filters and Sample Rate Conversion*, *YouTube*, 2021. [Online Video]. Available: <https://www.youtube.com/watch?v=zVM8NKXsboA>
- [5] “resample_poly — SciPy v1.16.2 Manual.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample_poly.html
- [6] “iirnotch — SciPy v1.16.2 Manual.” <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.iirnotch.html>
- [7] FIR decimator in MATLAB. Available: https://www.mathworks.com/help/dsp/ref/dsp.firdecimator-system-object.html?utm_source=chatgpt.com
- [8] *FPGA and DSP Ep. 4: Polyphase Filters*, *YouTube Channel: Engineering Mindset*, 2022. [Online Video]. Available: <https://www.youtube.com/watch?v=06XRSkkfhek>
- [9] “How many taps does an FIR filter need?,” *DSP Stack Exchange*, 2016. [Online]. Available: <https://dsp.stackexchange.com/questions/31066/how-many-taps-does-an-fir-filter-need>
- [10] J. O. Smith III, “Introduction to Windowed FIR Filter Design,” *Center for Computer Research in Music and Acoustics (CCRMA), Stanford University*, 2011. [Online]. Available: https://ccrma.stanford.edu/~jos/WinFlt/WinFlt_2up.pdf
- [11] SciPy Developers, “iirnotch — SciPy v1.14.1 Reference Guide,” *SciPy Documentation*, 2024. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.iirnotch.html>
- [12] L. Wanhammar and T. Saramäki, Digital filters using MATLAB. 2020. doi: 10.1007/978-3-030-24063-9. <https://link.springer.com/book/10.1007/978-3-030-24063-9>
- [13] Multirate digital filters, filter banks, polyphase networks, and applications: a tutorial, *IEEE Journals & Magazine — IEEE Xplore*, Jan. 01, 1990. <https://ieeexplore.ieee.org/document/52200>

- [14] Phijurag, “Digital Signal Processing An Introduction D Sundararajan No waiting time,” Scribd. <https://www.scribd.com/document/911582073/Digital-Signal-Processing-An-Introduction-D-Sundararajan-No-Waiting-Time>